

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – PSEUDOCODE WRITING TASK

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO5 – Naturalization (BL4, BL6)	Assessment:	Assessment Tool 1 – Pseudocode Writing Task
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	<i>Construct MST using shortest path algorithms and traverse the graph to model and analyse realworld problem.</i>		
Student Name:	C.Mokshithareddy	Reg. No.:	192571054
Section / Batch:		Date:	31/08/2026

Code of Conduct

I

_____C.Mokshithareddy(192571054)_____

_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the Student: __Mokshitha.C._____

Instructions

- This test contains 5 Pseudocode Writing Task questions. Total: 100 Marks.
- When a student answer using pseudocode writing task should evaluate based on the ability to design clear, logical, and efficient pseudocode solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Minimum Spanning Tree	Kruskal's Algorithm	1	20
Graph Sorting	Topological Sort	2	20

Shortest Path Algorithm	MST & Dijkstra's Algorithm	3	20
Shortest Path Algorithm	Dijkstra's Algorithm	4	20
Minimum Spanning Tree	Prim's or Kruskal's Algorithm	5	20
GRAND TOTAL:		100 Marks	

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or nonfunctional code
Code Quality & Readability	20	Clean, modular, wellcommented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence
Faculty In-charge		Course Coordinator		Program Director	
Signature: _____		Signature: _____		Signature: _____	
Date: _____		Date: _____		Date: _____	

PSEUDOCODE WRITING TASK
[← Back](#)
QUESTIONS
Q1 ✓
Minimum Spanning Tree -
Kruskal's Algorithm
20 marks

Q2 ✓
Graph - Topological Sort
20 marks

Q3 ✓
Shortest Path Algorithm -
Minimum Spanning Tree &
Dijkstra's
20 marks

Q4 ✓
Shortest Path Algorithm -
Dijkstra's Algorithm
20 marks

Q5 ✓
Minimum Spanning Tree -
Prim's or Kruskal's Algorithm

Q1 Minimum Spanning Tree - Kruskal's Algorithm

20 marks

Write pseudocode for constructing an MST using Kruskal's Algorithm. A city planning department needs to connect multiple zones with underground fiber-optic cables. Each zone is represented as a vertex, and possible cable connections between zones are represented as edges with installation costs (weights).

Constraints:

- All zones must be connected
- Total installation cost must be minimized
- No redundant connections (no cycles) allowed
- The network must remain fully connected

Your Answer

KRUSKAL(G):
MST = empty set
sort all edges of G in increasing order of weight

for each vertex v in G:
MAKE-SET(v)

d. The network must remain fully connected

Your Answer

KRUSKAL(G):
MST = empty set
sort all edges of G in increasing order of weight

for each vertex v in G:
MAKE-SET(v)

 **Save Answer**
Next →

Your Answer

```
for each edge (u, v) in sorted edges:
    if FIND-SET(u) != FIND-SET(v):
        add (u, v) to MST
        UNION(u, v)

if number of edges in MST == n - 1:
```

 Save Answer

Next →

Your Answer

```
        UNION(u, v)

if number of edges in MST == n - 1:
    break

return MST
```

 Save Answer

Next →

SIMATS
ENGINEERING

SIMATS Online Programming Lab
DS PROGRAMMING

CHALAMALA MOKSHITHA
REDDY
192571054RA

PSEUDOCODE WRITING TASK

QUESTIONS

Q1
Minimum Spanning Tree -
Kruskal's Algorithm
20 marks

Q2
Graph - Topological Sort
20 marks

Q3
Shortest Path Algorithm -
Minimum Spanning Tree &
Dijkstra's
20 marks

Q4
Shortest Path Algorithm -
Dijkstra's Algorithm
20 marks

Q5
Minimum Spanning Tree -
Kruskal's Algorithm
20 marks

Q2 Graph - Topological Sort

20 marks

A DAG represents task dependencies. Write pseudocode for Topological Sort using DFS with cycle detection.
Constraints:
a. Detect cycle before sorting
b. Use DFS approach

Your Answer

TOPOLOGICAL_SORT(G):
visited[v] = false for every vertex v
path[v] = false for every vertex v
stack = empty

for each vertex v:

Save Answer

Next →

Your Answer

```
TOPOLOGICAL_SORT(G):  
    visited[v] = false for every vertex v  
    path[v] = false for every vertex v  
    stack = empty  
  
    for each vertex v:
```

Save Answer

Next →

Your Answer

```
for each vertex v:  
  if visited[v] == false:  
    if DFS(v) == true:  
      print "Cycle detected – Topological sort not possible"  
      return
```

 Save Answer

Next →

Your Answer

print vertices from stack in reverse order

```
DFS(v):  
  visited[v] = true  
  path[v] = true
```

 Save Answer

Next →

Your Answer

```
for each neighbor u of v:  
    if path[u] == true:  
        return true    // Cycle detected  
  
    if visited[u] == false:  
        if DFS(u) == true:  
            return true
```

 Save Answer

Next →

Your Answer

```
    if DFS(u) == true:  
        return true  
  
    path[v] = false  
    push v into stack  
    return false
```

 Save Answer

Next →

PSEUDOCODE WRITING TASK

← Back

QUESTIONS

Q1

Minimum Spanning Tree -
Kruskal's Algorithm
20 marks

Q2

Graph - Topological Sort
20 marks

Q3

Shortest Path Algorithm -
Minimum Spanning Tree &
Dijkstra's
20 marks

Q4

Shortest Path Algorithm -
Dijkstra's Algorithm
20 marks

Q5

Minimum Spanning Tree -
Prim's or Kruskal's Algorithm

Q3 Shortest Path Algorithm - Minimum Spanning Tree & Dijkstra's

20 marks

Write pseudocode integrating MST + Dijkstra for Smart transportation system.

Constraints:

- Use MST for infrastructure
- Use shortest path for routing

Your Answer

SMART_TRANSPORT(G, S, D)

- MST = KRUSKAL(G)
- Set distance of all vertices = ∞
- distance[S] = 0

Save Answer

Next →

Your Answer

SMART_TRANSPORT(G, S, D)

- MST = KRUSKAL(G)
- Set distance of all vertices = ∞
- distance[S] = 0

Save Answer

Next →

Your Answer

4. Set $\text{parent}[S] = \text{NULL}$
5. Put S into Priority Queue
6. While Priority Queue is not empty:
 u = Remove vertex with minimum distance

 Save Answer

Next →

Your Answer

For each edge (u, v) in MST:
 $\text{newDist} = \text{distance}[u] + \text{weight}(u, v)$

If $\text{newDist} < \text{distance}[v]$:
 $\text{distance}[v] = \text{newDist}$
 $\text{parent}[v] = u$
 Add v to Priority Queue

 Save Answer

Next →

Your Answer

Add v to Priority Queue

7. Print MST as minimum-cost infrastructure

8. Print path from S to D using parent[]

9. Print shortest distance = distance[D]

 Save Answer

Next →

Your Answer

8. Print path from S to D using parent[]

9. Print shortest distance = distance[D]

END

 Save Answer

Next →

PSEUDOCODE WRITING TASK

[← Back](#)

QUESTIONS

Q1
Minimum Spanning Tree -
Kruskal's Algorithm
20 marks

Q2
Graph - Topological Sort
20 marks

Q3
Shortest Path Algorithm -
Minimum Spanning Tree &
Dijkstra's
20 marks

Q4
Shortest Path Algorithm -
Dijkstra's Algorithm
20 marks

Q5
Minimum Spanning Tree -
Kruskal's Algorithm

Q4 Shortest Path Algorithm - Dijkstra's Algorithm

20 marks

Shortest Path with "Toll-Free" Voucher: You have a voucher that allows you to traverse one edge for free (weight = 0). Write pseudocode to find the shortest path from S to D utilizing this voucher optimally.

Your Answer

DIJKSTRA_VOUCHER(G, S, D):

 $\text{dist}[v][0] = \infty$ $\text{dist}[v][1] = \infty$ $\text{dist}[S][0] = 0$ [Save Answer](#)[Next →](#)

Your Answer

DIJKSTRA_VOUCHER(G, S, D):

 $\text{dist}[v][0] = \infty$ $\text{dist}[v][1] = \infty$ $\text{dist}[S][0] = 0$ [Save Answer](#)[Next →](#)

Your Answer

```
Priority Queue ← (0, S, 0)
```

```
while Priority Queue is not empty:
```

```
    (cost, u, used) = extract_min()
```

 Save Answer

Next →

Your Answer

```
for each edge (u, v, w):
```

```
    // Normal travel
```

```
    newCost = cost + w
```

```
    if newCost < dist[v][used]:
```

```
        dist[v][used] = newCost
```

 Save Answer

Next →

Shortest Path with 100% Free Voucher. You have a voucher that allows
pseudocode to find the shortest path from S to D utilizing this voucher

Your Answer

```
insert(v, newCost, used)
```

```
// Use voucher on this edge
```

```
if used == 0:
```

```
    if cost < dist[v][1]:
```

```
        dist[v][1] = cost
```

 Save Answer

Next →

Your Answer

```
if used == 0:
```

```
    if cost < dist[v][1]:
```

```
        dist[v][1] = cost
```

```
    insert(v, cost, 1)
```

```
return min(dist[D][0], dist[D][1])
```

 Save Answer

Next →

PSEUDOCODE WRITING TASK

[← Back](#)

QUESTIONS

Q1 ✓
Minimum Spanning Tree -
Kruskal's Algorithm
20 marks

Q2 ✓
Graph - Topological Sort
20 marks

Q3 ✓
Shortest Path Algorithm -
Minimum Spanning Tree &
Dijkstra's
20 marks

Q4 ✓
Shortest Path Algorithm -
Dijkstra's Algorithm
20 marks

Q5 ✓
Minimum Spanning Tree -
Prim's or Kruskal's Algorithm

Q5 Minimum Spanning Tree - Prim's or Kruskal's Algorithm

20 marks

Second Best Spanning Tree: Design pseudocode to find the "Second Best" MST.

Hint: Find the MST first, then for every edge in the MST, try removing it and finding the next best edge to replace it.

Your Answer

SECOND_BEST_MST(G):

```
MST = KRUSKAL(G)
best = weight(MST)
secondBest = ∞
```

[Save Answer](#)

Your Answer

SECOND_BEST_MST(G):

```
MST = KRUSKAL(G)
best = weight(MST)
secondBest = ∞
```

[Save Answer](#)

Your Answer

for each edge e in MST:

Remove e from G

$T = \text{KRUSKAL}(G)$

if T is connected:

 Save Answer

Your Answer

if T is connected:

$\text{cost} = \text{weight}(T)$

if $\text{cost} > \text{best}$ AND $\text{cost} < \text{secondBest}$:

$\text{secondBest} = \text{cost}$

 Save Answer

Your Answer

```
cost = weight(T)
```

```
if cost > best AND cost < secondBest:
```

```
    secondBest = cost
```

```
return secondBest
```

 Save Answer